

Rapport PPN Semestre 1

Filière : M1 Calcul Haute Performance et Simulation

Années : 2025-2026

**Sujet : Path tracing : méthodes de
Monte Carlo pour la synthèse d'images**

Rédigé par :
Nolwen DOLLÉANS
Prince Marcel MASSAMBA
Théo BAUER

Encadré par :
Mr. Aurélien DELVAL

Table des matières

1	Introduction	3
1.1	Qu'est-ce que le Path Tracing ?	3
1.1.1	Le Ray Casting	3
1.2	Comparaison avec des technique existantes	3
1.3	Limites du Path Tracing	3
1.4	Données utilisées	4
2	Définition de l'écran	5
2.1	Aspect Ratio	5
2.2	Obtention des vecteurs des rayons	7
3	Définition d'objets dans l'espace	9
3.1	Équation d'un rayon	9
3.2	Équation d'une sphère	9
3.3	Intersection d'un rayon sur une sphère	9
4	AABB	10
5	Équation de rendu	13
5.1	Les matériaux Lambertiens	13
5.2	Approximation avec la méthode Monte-Carlo	14
5.2.1	Méthode d'échantillonnage Cosine-Weighted	14
5.2.2	Application de la Méthode de Monte-Carlo pour l'équation de rendu	14
5.3	Implémentation	15
6	Résultats	16
6.1	Convergence	16
6.2	Analyse de performances	17
6.2.1	Protocole de mesure	17
6.2.2	Discussion de l'implémentation actuelle	17
6.2.3	Possibles améliorations	17
7	Annexes	18
7.1	Démonstration de l'équation de rendu	18
7.1.1	Cas Général	18
7.1.2	Matériaux Lambertiens	18
7.2	Démonstration sur les matrices de rotations	19
7.3	Parité du nombres de pixels dans la formation d'un écran	21
8	Bibliographie	23

1 Introduction

La modélisation en 3 dimensions est devenue incontournable depuis des décennies. Elle est en effet présente dans différents domaines et a des applications variées comme l'animation, les jeux vidéos ou encore la simulation réaliste. Cette manière de représenter notre monde dans un cadre réaliste provient notamment de la simulation de trajectoires des photons.

Pour ce projet, nous allons nous concentrer sur la technique du Path Tracing, combinant la technique du Ray Tracing, ainsi qu'une approche aléatoire pour résoudre des problèmes déterministes par la méthode de Monte-Carlo.

1.1 Qu'est-ce que le Path Tracing ?

Le **Path Tracing** est une technique de rendu 3D se basant sur la méthode de Monte-Carlo pour approximer l'équation de transport de la lumière. Le principe est d'échantillonner aléatoirement un grand nombre de chemins lumineux entre la caméra et les différentes sources de lumière présentes dans la scène. Un exemple est présent dans le [résultat 6.1](#).

1.1.1 Le Ray Casting

Le Path Tracing se base sur le **Ray Casting**. Le ray casting est une technique de rendu visuel qui a pour principe d'afficher des images avec une illusion de profondeur / 3D. Cette technique a été popularisée par le studio ID Software, qui l'a entre autre utilisé pour Wolfenstein 3D, DOOM et Quake. Elle fut souvent utilisée pour du temps réel, car elle était très peu coûteuse en termes de calcul pour les machines de l'époque.

L'environnement est représenté de la manière suivante : une caméra à partir duquel on balance un rayon par pixel. Pour chaque pixel tracer, on calcule l'intersection de chaque objet touché par le rayon. Ensuite, on affiche le pixel avec la distance la plus petite.

Cette technique apporte quelques inconvénients, d'où le fait qu'elle soit utilisée en complément d'autres techniques de rendu. Elle ne permet pas d'effectuer du rendu sur des objets 3D complexes. Pour du temps réel, nous sommes limité sur une rotation par rapport à l'axe de Y. Nous ne pouvons pas faire du rendu volumétrique, i.e rendu d'un effet de flamme ou liquide. Pour du temps réel, nous sommes limité sur une rotation par rapport à l'axe de Y. On ne gère pas les rebonds.

1.2 Comparaison avec des technique existantes

- **Rasterization** : technique de rendu 3D utilisée pour convertir des objets 3D en une image 2D affichable à l'écran. Elle fonctionne triangle par triangle plutôt que pixel par pixel. Très rapide, optimisée pour des scènes avec des millions de triangles et avec des exigences en temps-réel. Mais les ombres/éclairages indirects, réflexions complexes et phénomènes optiques réalistes restent approximatifs.
- **Ray tracing** : lance des rayons partants de la caméra pour simuler les ombres, réflexions et occultations ne traitant alors pas toute la scène, mais uniquement les trajectoires des photons qui seront interceptés. Cela donne un rendu plus fidèle que le rasterization mais souvent limité à quelques rebonds ou approximations car plus coûteuse en temps et en énergie.

De nos jours, la technique de rendu la plus populaire est la rasterization. Bien que moins précise visuellement, la rasterization demeure dominante dans les moteurs temps réel.

1.3 Limites du Path Tracing

Le Path-Tracing étant un modèle, il nous faut de poser des fonctionnements qui ne font pas diverger la simulation de la réalité. Les chemins lumineux parcourent un ligne droite entre chacuns des rebonds, sans que d'autres éléments puissent les perturber ; en particulier d'autres chemins lumineux ou un effet de brume ou fumée. De plus, les rayonnements ne font pas appel aux propriétés des ondes électromagnétiques qui sont connus pour les photons. D'autres effets spécifiques ne sont pas simulables, telles

que la fluorescence ou la polarisation. Cependant ces limitations ne représentent pas une vraie limite du modèle, leur impact est mineur sur le rendu.

Pour les limites liées à l'utilisation du Path-Tracing, il y a premièrement le problème de son implémentation pour un rendu en temps réel, le matériel nécessaire pour son implémentation est coûteux et donc le rendu devient dépendant de la capacité à traiter les images.

Le Path-Tracing comprend de plus un test pour d'éventuelles intersections de chemins lumineux, ce qui demande des performances, particulièrement pour des modèles 3D complexes.

1.4 Données utilisées

Il nous faut de poser les conditions dans lesquelles nous allons traiter le reste du projet. Tout d'abord, nous avons une technique de rendu 3D, c'est pourquoi l'ensemble des représentations d'objets se trouvent dans $\mathbb{R}^3$, nous posons avant tout trois vecteurs $\vec{r}_0, \vec{u}_0, \vec{d}_0 \in \mathbb{R}^3$ comme la base canonique : $(\vec{r}_0, \vec{u}_0, \vec{d}_0) = \left(\begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \right)$; ayant choisi visuellement : $\vec{r}_0$ pour "right", $\vec{u}_0$ pour "up" et $\vec{d}_0$ pour "direction" vers où nous pointons.

Nous définissons ensuite toute norme utilisée comme la norme 2 euclidienne notée telle que :

$$\begin{aligned} \|\cdot\| : \mathbb{R}^3 &\rightarrow \mathbb{R} \\ \begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} &\mapsto \sqrt{a_1^2 + a_2^2 + a_3^2} \end{aligned}$$

De même, le produit scalaire sera :

$$\begin{aligned} \cdot : \mathbb{R}^3, \mathbb{R}^3 &\rightarrow \mathbb{R} \\ \begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} \cdot \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix} &\mapsto a_1 b_1 + a_2 b_2 + a_3 b_3 \end{aligned}$$

Finalement, le produit vectoriel est :

$$\begin{aligned} \cdot \wedge \cdot : \mathbb{R}^3, \mathbb{R}^3 &\rightarrow \mathbb{R}^3 \\ \begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} \wedge \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix} &\mapsto \begin{pmatrix} a_2 b_3 - a_3 b_2 \\ a_1 b_3 - a_3 b_1 \\ a_1 b_2 - a_2 b_1 \end{pmatrix} \end{aligned}$$

2 Définition de l'écran

Nous définissons notre écran comme la formation d'une grille carré de pixels, la grille est de largeur w (width) et de hauteur h (height) en nombre de pixels. Chaque pixel a un centre qui intersecte la direction de chacun des rayons sortant de la caméra. La caméra est définie au point $\mathbf{E} \in \mathbb{R}^3$ et dans la base $(\vec{r}, \vec{u}, \vec{d})$ telle qu'explicitée dans l'[Annexe 7.2](#). Le centre de l'écran est le point $\mathbf{C} \in \mathbb{R}^3$ résultant de $\mathbf{E} + \vec{d}$, il se trouve ainsi à distance 1 de la caméra. Nous définissons finalement un angle $\theta \in [0, \pi]$ représentant la "Field Of View" (FOV). Généralement, l'oeil humain est au repos avec un FOV entre 50 et 60 degrés.

Notons ici que l'écran forme un plan parallèle à $(\vec{r}, \vec{u})$ et qu'il dépend donc de l'orientation de la caméra. Exemple : Le résultat finale d'un écran tel que nous le définissons dans cette partie se visualise de la manière suivante :

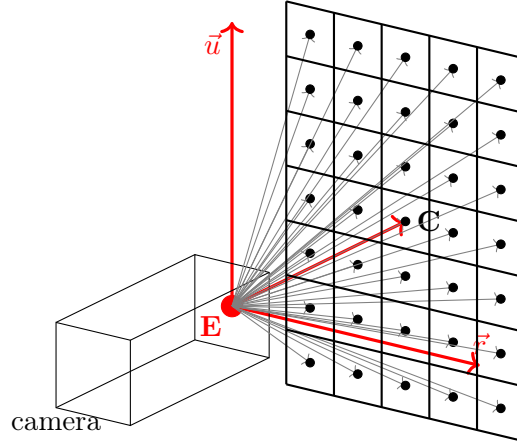


FIGURE 2.1 – Écran dans $\mathbb{R}^3$ par rapport à la caméra
 $(w, h) = (5, 7)$ et $\theta = \frac{\pi}{3}$

2.1 Aspect Ratio

Or nous nous rendons compte que le nombre de pixels en width et en height ne sont pas toujours égaux, donc pour un même FOV nous obtenons un effet d'étirement de l'image. Nous considérons l'angle θ autour de $\vec{r}$ et de $\vec{u}$:

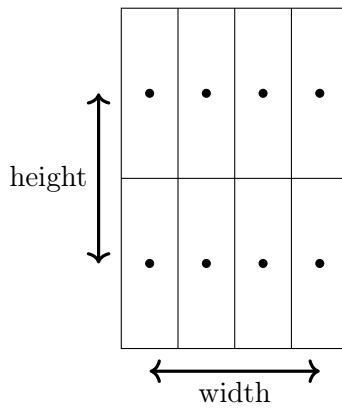


FIGURE 2.2 – Écran avant de résoudre le problème lié à l'aspect ratio
 $(w, h) = (4, 2)$

Les pixels ne sont pas eux-même carrés et lorsque nous représentont des objets sur ce type d'écran, nous voyons l'effet suivant :

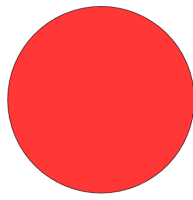


Figure 2.3.A - 480×480 pixels

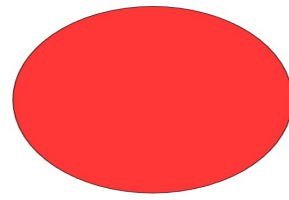


Figure 2.3.B - 720×480 pixels

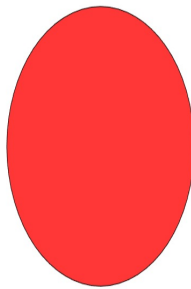


Figure 2.3.C - 480×720 pixels

FIGURE 2.3 – Différentes résolution d'une même image

L'aspect ratio se résume au quotient entre la width-1 et la height-1 de l'image en nombre de pixels :

$AR = \frac{w-1}{h-1}$. Pour les figures ci-dessus :

- $AspectRatio_1 = \frac{480-1}{480-1} = 1$.
- $AspectRatio_2 = \frac{720-1}{480-1} = 1.501 > 1$.
- $AspectRatio_3 = \frac{480-1}{720-1} = 0.666 < 1$.

Afin de corriger ce problème, nous multiplions la width par l'aspect ratio :

En reprenant la [figure 2.2](#) :

$$AspectRatio = \frac{4-1}{2-1} = 3$$

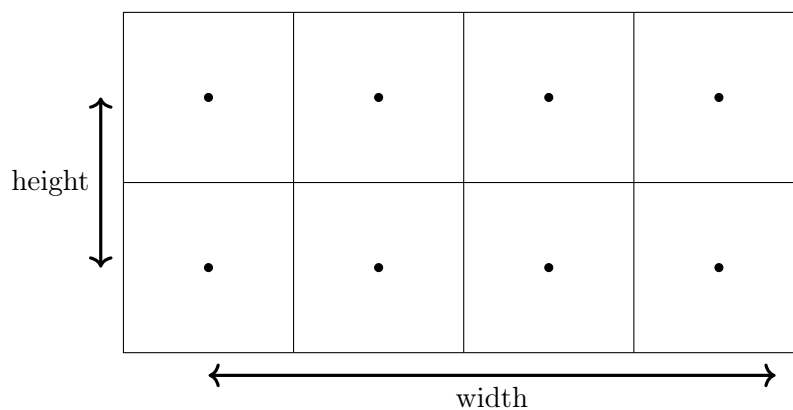


FIGURE 2.4 – Écran avant de résoudre le problème lié à l'aspect ratio
 $(w, h) = (4, 2)$

2.2 Obtention des vecteurs des rayons

Nous voulons dans cette partie déterminer comment obtenir chacun des vecteurs dirigeant les rayons en fonction du nombre des pixels (w, h) et de l'angle θ .

Nous posons $\forall (i, j) \in \mathbb{N}$ tels que $0 \leq i \leq w - 1, 0 \leq j \leq h - 1, P_{i,j} \in \mathbb{R}^3$ chacun des centres des pixels. En reprenant l'exemple de la [figure 2.1](#)

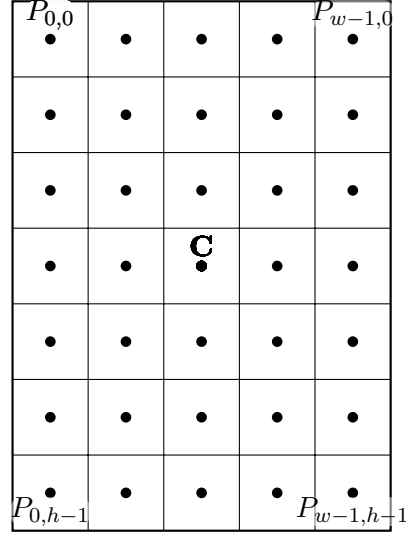


FIGURE 2.5 – Définition de la notation des pixels sur l'écran
 $(w, h) = (5, 7)$

Par simples règles de trigonométrie sur des triangles rectangles, nous savons que :

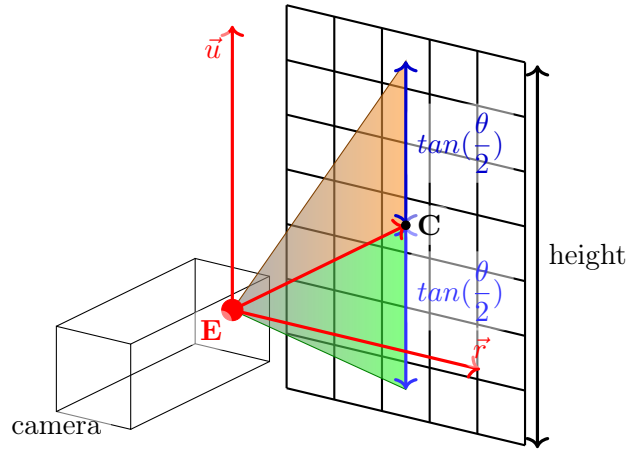
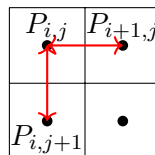


FIGURE 2.6 – La hauteur de l'écran déterminée par la FOV est $2 \tan(\frac{\theta}{2})$
Vrai $\forall (w, h) \in \mathbb{R}$ et $\forall \theta \in [0, \pi]$

Remarquons que le résultat de cette distance sur la width est $2 \frac{w-1}{h-1} \tan(\frac{\theta}{2})$, il suffit de multiplier par l'aspect ratio.

Nous pouvons maintenant chercher la distance séparant deux centres de pixels adjacents, c'est-à-dire la distance :



Nous trouvons que cette distance est égale à $\frac{2 \tan(\frac{\theta}{2})}{w-1}$

Nous savons que $\mathbf{C} = \mathbf{E} + \vec{d}$ et cherchons à déterminer les coordonnées de $P_{0,0}$:

Nous prenons compte des vecteurs $\vec{d}_r, \vec{d}_u \in \mathbb{R}^3$ représentant la distance entre $\mathbf{C}$ et les centres des pixels aux extrémités de l'écran (cf [Annexe 7.3](#)). Ainsi :

$$\begin{aligned}\vec{d}_r &= -2 \left(\frac{w-1}{h-1} \right) \tan\left(\frac{\theta}{2}\right) \frac{w-1}{2(w-1)} \vec{r} = -\left(\frac{w-1}{h-1} \right) \tan\left(\frac{\theta}{2}\right) \vec{r} \\ \vec{d}_u &= 2 \tan\left(\frac{\theta}{2}\right) \frac{h-1}{2(h-1)} \vec{u} = \tan\left(\frac{\theta}{2}\right) \vec{u}\end{aligned}$$

Nous en déduisons que les coordonnées de $P_{0,0}$ sont donc :

$$\begin{aligned}P_{0,0} &= \vec{d}_r + \vec{d}_u + \mathbf{C} = -\left(\frac{w-1}{h-1} \right) \tan\left(\frac{\theta}{2}\right) \vec{r} + \tan\left(\frac{\theta}{2}\right) \vec{u} + \mathbf{C} \\ P_{0,0} &= \begin{pmatrix} -\left(\frac{w-1}{h-1} \right) \tan\left(\frac{\theta}{2}\right) \\ 0 \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ \tan\left(\frac{\theta}{2}\right) \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \\ P_{0,0} &= \begin{pmatrix} -\left(\frac{w-1}{h-1} \right) \tan\left(\frac{\theta}{2}\right) \\ \tan\left(\frac{\theta}{2}\right) \\ 1 \end{pmatrix}\end{aligned}$$

De cette manière, nous pouvons finalement exprimer $P_{i,j}$ en fonction de $(w, h), \theta$ et $P_{0,0}$:

$$\begin{aligned}\forall (i, j) \in \mathbb{N} \text{ tels que } 0 \leq i \leq w-1, 0 \leq j \leq h-1 : \\ P_{i,j} &= P_{0,0} + i \frac{2}{w-1} \tan\left(\frac{\theta}{2}\right) \vec{r} - j \frac{2}{h-1} \tan\left(\frac{\theta}{2}\right) \vec{u} \\ P_{i,j} &= \begin{pmatrix} -\left(\frac{w-1}{h-1} \right) \tan\left(\frac{\theta}{2}\right) \\ \tan\left(\frac{\theta}{2}\right) \\ 1 \end{pmatrix} + \begin{pmatrix} i \frac{2}{w-1} \tan\left(\frac{\theta}{2}\right) \\ 0 \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ -j \frac{2}{h-1} \tan\left(\frac{\theta}{2}\right) \\ 0 \end{pmatrix} \\ P_{i,j} &= \begin{pmatrix} \tan\left(\frac{\theta}{2}\right) \times \left(i \frac{2}{w-1} - \frac{w-1}{h-1} \right) \\ \tan\left(\frac{\theta}{2}\right) \times \left(1 - j \frac{1}{h-1} \right) \\ 1 \end{pmatrix}\end{aligned}$$

Et donc les vecteurs directeurs de chacun de ces points partant de la caméra sont :

$\forall (i, j) \in \mathbb{N}$ tels que $0 \leq i \leq w-1, 0 \leq j \leq h-1$, $p_{i,j} \in \mathbb{R}^3$, $p_{i,j} = \overrightarrow{\mathbf{E}P_{i,j}}$ qui en particulier coïncide avec les coordonnées de $P_{i,j}$.

Nous voulons cependant normaliser ces vecteurs, nous définissons donc :

$\forall (i, j) \in \mathbb{N}$ tel que $0 \leq i \leq w-1, 0 \leq j \leq h-1$, $r_{i,j} \in \mathbb{R}^3$, $r_{i,j} = \frac{p_{i,j}}{\|p_{i,j}\|}$. Les $r_{i,j}$ sont les rayons normalisés sortant de la caméra dont nous connaissons maintenant les coordonnées.

3 Définition d'objets dans l'espace

3.1 Équation d'un rayon

Pour tout vecteur $\vec{d} \in \mathbb{R}^3$ d'origine $O \in \mathbb{R}^3$ dans la direction $T \in \mathbb{R}^3$, on a $\vec{d} = \overrightarrow{(O + T)}$ qui donne l'équation de rayon :

$$\forall t \in \mathbb{R}, \vec{p}(t) = O + \vec{d}t = \begin{pmatrix} o_x \\ o_y \\ o_z \end{pmatrix} + \begin{pmatrix} d_x \\ d_y \\ d_z \end{pmatrix} t = \begin{pmatrix} o_x + d_x t \\ o_y + d_y t \\ o_z + d_z t \end{pmatrix}.$$

3.2 Équation d'une sphère

Pour une position $C \in \mathbb{R}^3$ et un rayon $r \in \mathbb{R}_+$ on définit une sphère $s = (C, r)$ d'équation :

$$\begin{aligned} s &= \{(x, y, z) \in \mathbb{R}^3 \mid (x - c_x)^2 + (y - c_y)^2 + (z - c_z)^2 = r^2\} \\ \Leftrightarrow s &= \{(x, y, z) \in \mathbb{R}^3 \mid (x - c_x)^2 + (y - c_y)^2 + (z - c_z)^2 - r^2 = 0\}. \end{aligned}$$

3.3 Intersection d'un rayon sur une sphère

Dans un premier temps, on injecte $\vec{p}(t)$ dans l'équation de $s(x, y, z)$, on obtient :

$$\begin{aligned} s(\vec{p}(t)) &= (p_x(t) - c_x)^2 + (p_y(t) - c_y)^2 + (p_z(t) - c_z)^2 - r^2 \\ &= (o_x + d_x t - c_x)^2 + (o_y + d_y t - c_y)^2 + (o_z + d_z t - c_z)^2 - r^2 \\ &= (d_x^2 + d_y^2 + d_z^2)t^2 + 2(o_x d_x + o_y d_y + o_z d_z - c_x d_x - c_y d_y - c_z d_z)t \\ &\quad + (o_x^2 + o_y^2 + o_z^2 - c_x^2 - c_y^2 - c_z^2 - r^2) \\ &= \|\vec{d}\|^2 t^2 + 2\vec{d} \cdot \overrightarrow{(O - C)}t + \|\overrightarrow{(O - C)}\|^2 - r^2 \end{aligned}$$

Soit $s(\vec{p}(t))$ une équation quadratique, avec $a = \|\vec{d}\|^2$, $b = 2\vec{d} \cdot \overrightarrow{(O - C)}$, $c = \|\overrightarrow{(O - C)}\|^2 - r^2$

On doit résoudre $at^2 + bt + c = 0$.

Pour un discriminant $\Delta = b^2 - 4ac$:

- Aucune intersection si $\Delta < 0$;

- Une intersection si $\Delta = 0 \Rightarrow x_1 = \frac{-b}{2a}$;

- Deux intersections si $\Delta > 0 \Rightarrow x_1 = \frac{-b - \sqrt{\Delta}}{2a}$, $x_2 = \frac{-b + \sqrt{\Delta}}{2a}$.

Algorithm 1 intersection Sphere

Require: Sphere, Ray and $hit \in \mathbb{R}^3$

```
 $a = Ray.direction * Ray.direction$   
 $b = 2 * Ray.direction * (Ray.position - Ray.direction)$   
 $c = (Ray.position - Ray.direction)^2 - radius^2$   
 $\Delta = B^2 - 4AC$   
if  $\Delta < 0$  then  
    return false  
else if  $\Delta == 0$  then  
    return false  
else if  $\Delta > 0$  then  
     $t = \frac{-b}{2a}$   
else if  $\Delta > 0$  then  
     $t = \frac{-b - \sqrt{\Delta}}{2a}$   
end if  
 $hit = Ray.position + Ray.direction * t$   
return true
```

Complexité spatial : $O(1)$

Complexité temporel : $O(1)$

4 AABB

Un AABB (Axis-Aligned Bounding Box) est un pavé droit qui nous sert d'espace de rebond pour nos rayon.

Nous pouvons le définir par deux points $B_{min} \in \mathbb{R}^3$, $B_{max} \in \mathbb{R}^3$ représentant deux sommets aux extrémités du cube, chacune des arêtes sont ensuite colinéaires à un des vecteurs de la base canonique.

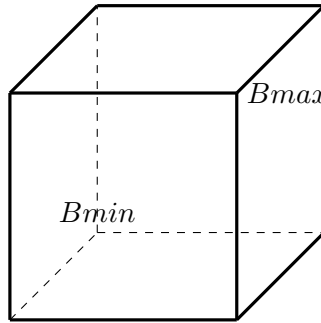


FIGURE 4.1 – AABB en fonction de B_{min} et B_{max}

Nous traçons un rayon défini par $\vec{r}(t) = o + \vec{d}t$.

Dans le cas où il y a intersection :

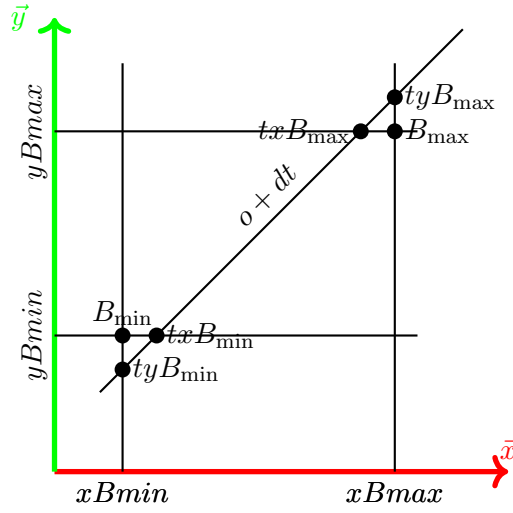


FIGURE 4.2 – Points d'intersection entre un rayon $o + dt$ et l'AABB

Nous avons 6 équations linéaire à résoudre :

$$\begin{cases} xB_{\min} = o_x + d_x txB_{\min} \implies txB_{\min} = (xB_{\min} - o_x)/d_x \\ yB_{\min} = o_y + d_y tyB_{\min} \implies tyB_{\min} = (yB_{\min} - o_y)/d_y \\ zB_{\min} = o_z + d_z tzB_{\min} \implies tzB_{\min} = (zB_{\min} - o_z)/d_z \\ xB_{\max} = o_x + d_x txB_{\max} \implies txB_{\max} = (xB_{\max} - o_x)/d_x \\ yB_{\max} = o_y + d_y tyB_{\max} \implies tyB_{\max} = (yB_{\max} - o_y)/d_y \\ zB_{\max} = o_z + d_z tzB_{\max} \implies tzB_{\max} = (zB_{\max} - o_z)/d_z \end{cases}$$

Ensuite, pour déterminer s'il y a intersection, on doit comparer la valeur maximale des solutions $tB_{i_{\min}}$ à la valeur minimale des solutions $tB_{i_{\max}}$:

- $t_{\min} = \max(txB_{\min}, tyB_{\min}, tzB_{\min})$
- $t_{\max} = \min(txB_{\max}, tyB_{\max}, tzB_{\max})$
- Si il y a intersection, alors $t_{\min} < t_{\max}$

Dans le cas où il n'y a pas d'intersection :

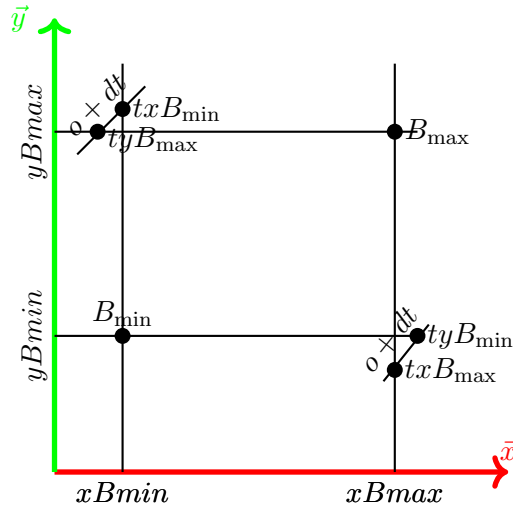


FIGURE 4.3 – Représentation des points n'intersectant pas l'AABB

Comme on peut le voir, $tyB_{\max} < txB_{\min}$ et $txB_{\max} < tyB_{\min}$, il ne peut pas y avoir une intersection.

Algorithm 2 intersection AABB

Require: AABB, Ray and $hit \in \mathbb{R}^3$

$$tmin = \frac{bmin(0) - Ray.position(0)}{Ray.direction(0)}$$

$$tmax = \frac{bmax(0) - Ray.position(0)}{Ray.direction(0)}$$

if $tmin > tmax$ **then**

 swap($tmin$, $tmax$)

end if

for $i = 1 : 3$ **do**

$$t1 = \frac{bmin(i) - Ray.position(i)}{Ray.direction(i)}$$

$$t2 = \frac{bmax(i) - Ray.position(i)}{Ray.direction(i)}$$

if $t1 > t2$ **then**

 swap($tmin$, $tmax$)

end if

$tmin = \max(tmin, t1)$

$tmax = \min(tmax, t2)$

if $tmin > tmax$ **then**

 return *false*

end if

end for

hit = Ray.position + Ray.direction * $tmax$

return *true*

Complexité spatial : $O(1)$

Complexité temporel : $O(1)$

5 Équation de rendu

L'équation de rendu représente l'impact de la radiance de la lumière sur une surface. En effet, elle va nous permettre de déterminer la manière dont la lumière se reflète sur un objet.

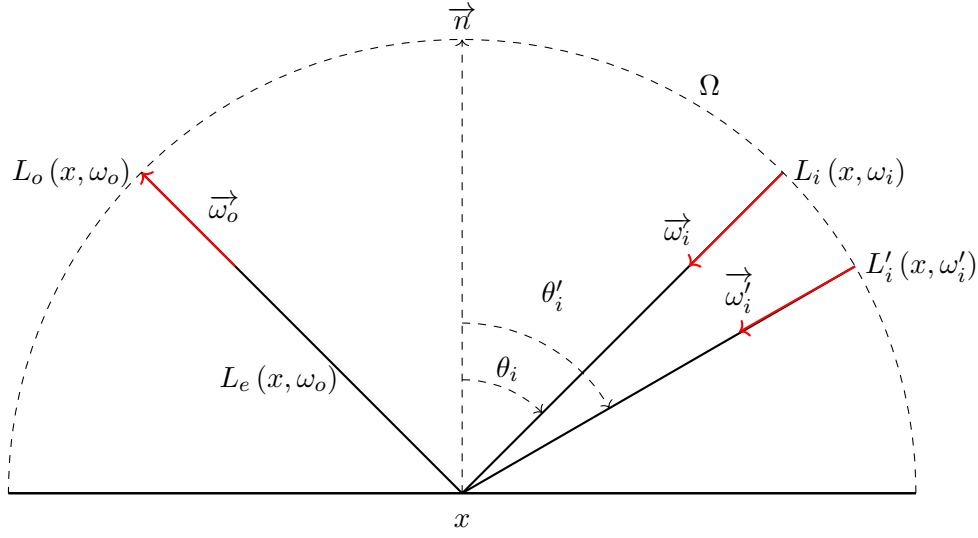


FIGURE 5.1 – Schéma de la réflexion total des rayons de directions ω_i résultant en un rayon de direction ω_o sur le point x d'une surface.

L'équation de rendu se définit comme suit (cf démonstration en [Annexe 7.1](#)) :

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_{\Omega} f(x, \omega_o, \omega_i) L_i(x, \omega_i) \cos(\theta_i) d\omega_i$$

avec :

- $\vec{n}$: La normale de la surface au point x .
- Ω : L'hémisphère centré autour de la normale $\vec{n}$.
- x : Le point de $\mathbb{R}^3$ d'intersection entre les rayons incidents et la surface de l'objet considéré.
- $\vec{\omega}_o$: Direction du rayon réfléchi dans Ω sur la surface au point x .
- $\vec{\omega}_i$: Directions des rayons incidents dans Ω sur la surface au point x .
- L_o : La radiance réfléchi sur la surface au point x de direction $\vec{\omega}_o$.
- L_e : La radiance émise par la surface au point x de direction $\vec{\omega}_o$.
- L_i : Les radiances des rayons incidents sur la surface au point x de direction $\vec{\omega}_i$.
- f : Le BRDF qui représente la manière dont le matériaux reflète la lumière.

5.1 Les matériaux Lambertiens

Cette équation n'est pas résoluble analytiquement. Pour faciliter les calculs et l'implémentation de notre path tracing, nous allons nous prendre le cas des matériaux dit "Lambertiens".

Ce type de matériaux inclut tous les matériaux de surface rugueuse qui n'émettent pas dans la lumière visible. Nous pouvons donc simplifier l'équation de rendu avec (cf démonstration [Annexe 7.1](#)) :

$$L_o(x, \omega) = \frac{\rho}{\pi} \int_{\Omega} L_i(x, \omega_i) \cos(\theta_i) d\omega_i$$

Même avec cette équation simplifiée, il est impossible de la résoudre analytiquement. C'est pour cela que nous allons utiliser une méthode stochastique pour pouvoir obtenir une solution convergeant vers la solution vraie : la méthode Monte-Carlo.

5.2 Approximation avec la méthode Monte-Carlo

La méthode de Monte-Carlo est une méthode itérative permettant de passer d'une intégrale à une somme d'échantillons aléatoires pondérée par une loi de probabilité notée *pdf* (Probability Density Function). Elle sera ici utilisée pour approximer la lumière reçue en un point en remplaçant l'intégrale par une somme d'échantillons aléatoires, chacun pondéré par une loi de probabilité par la *pdf*.

Ainsi, nous avons l'approximation suivante : Soit g , une fonction telle que : $g : \begin{cases} \mathbb{R} \mapsto \mathbb{R} \\ x \mapsto g(x) \end{cases}$:

$$I = \int_{\Omega} g(x) dx \approx \frac{1}{N} \sum_k^N \frac{g(x_k)}{pdf(x_k)}$$

5.2.1 Méthode d'échantillonnage Cosine-Weighted

Pour déterminer la direction des rayons aléatoires, nous choisissons un échantillonnage dit "Cosine-weighted". Il consiste à se placer dans le repère local orthonormé (*tangente*, *bitangente*, *normale*) au point d'intersection (cf. figure 5.2), pour ensuite construire le rayon aléatoire.

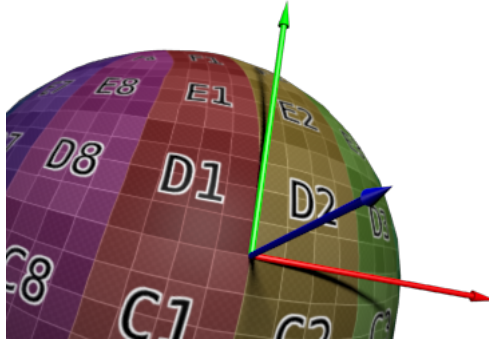


FIGURE 5.2 – Représentation de la base locale au point d'intersection du rayon sur la sphère avec la normale (en bleu), la bitangente (en vert) et la tangente (en rouge).

Source : <https://www.opengl-tutorial.org/fr/intermediate-tutorials/tutorial-13-normal-mapping/>

Ce mode d'échantillonnage se fait de la manière suivante :

- _ Génération de 2 nombres aléatoires $\xi_1 \in \mathbb{N}$ et $\xi_2 \in \mathbb{N}$
- _ Nous les passons en coordonnées polaires : $\begin{cases} r = \sqrt{\xi_1} \\ \theta = 2\pi\xi_2 \end{cases}$
- _ Nous passons les coordonnées polaires dans la base locale : $\begin{cases} x = r \cos \theta \\ y = r \sin \theta \\ z = \sqrt{1 - r^2} \end{cases}$

Ainsi, on construit la direction du rayon incident ω_i de la manière suivante dans la base $(\vec{r}_0, \vec{u}_0, \vec{d}_0)$: $\omega_i = x \cdot \text{tangente} + y \cdot \text{bitangente} + z \cdot \text{normale}$.

L'avantage de cette méthode d'échantillonnage est que la *pdf* sera : $pdf = \frac{\cos \theta_i}{\pi}$. C'est pour cette raison qu'elle est majoritairement utilisée dans le Path Tracing.

5.2.2 Application de la Méthode de Monte-Carlo pour l'équation de rendu

L'équation de rendu avec la méthode de Monte-Carlo s'écrit de la façon suivante :

$$L_o(x, \omega_o) = \frac{\rho}{\pi} \sum_{i=1}^N \frac{L_i(x, \omega_i) \cos(\theta_i)}{pdf(\omega_i)}$$

$$L_o(x, \omega_o) = \rho \sum_{i=1}^N L_i(x, \omega_i)$$

Ainsi, nous avons pu bien simplifier l'équation de rendu en choisissant le bon échantillonnage et en travaillant sur les bons matériaux.

5.3 Implémentation

Pour implémenter l'équation de rendu, nous allons utiliser la parité de la radiance (c'est-à-dire $L(x, \omega) = L(x, -\omega)$) afin de propager les rayons de la caméra vers la source de lumière plutôt que l'inverse, qui serait le phénomène physique naturel.

Le choix que nous avons fait pour notre première version naïve est la suivante : la somme $\rho \sum_{i=1}^N L_i(x, \omega_i)$ est représentée dans notre code comme étant une fonction récursive. En effet, pour chaque rebond, nous allons sommer $\rho L_i(x, \omega_i)$ et retourner le résultat pour chaque échantillon de pixel. La fonction sera de la manière suivante :

Algorithm 3 Échantillonnage récursif d'un rayon (RaySampling)

Require: Rayon r , scène S , caméra cam , profondeur d , profondeur max $d_{\max}$

Ensure: Couleur incidente L

```

1: Initialiser  $L \leftarrow (0, 0, 0)$ 
2:  $black \leftarrow (0, 0, 0)$ 
3: if  $d = d_{\max}$  then
4:   return  $black$ 
5: end if
6: if pas d'intersection de  $r$  avec la scène then
7:   return couleur de fond de la scène
8: end if
9: if l'objet est émissif then
10:   $L \leftarrow intensite \cdot couleur\_objet$ 
11:  return  $L$ 
12: end if
13: Générer un rayon réfléchi  $r_{new}$  dans l'hémisphère selon une loi Cosine-Weighted
14:  $L_{ref} \leftarrow RAYSAMPLING(r_{new}, S, cam, d + 1, d_{\max})$ 
15:  $L \leftarrow L + albedo \cdot L_{ref}$ 
16: return  $L$ 

```

6 Résultats

6.1 Convergence

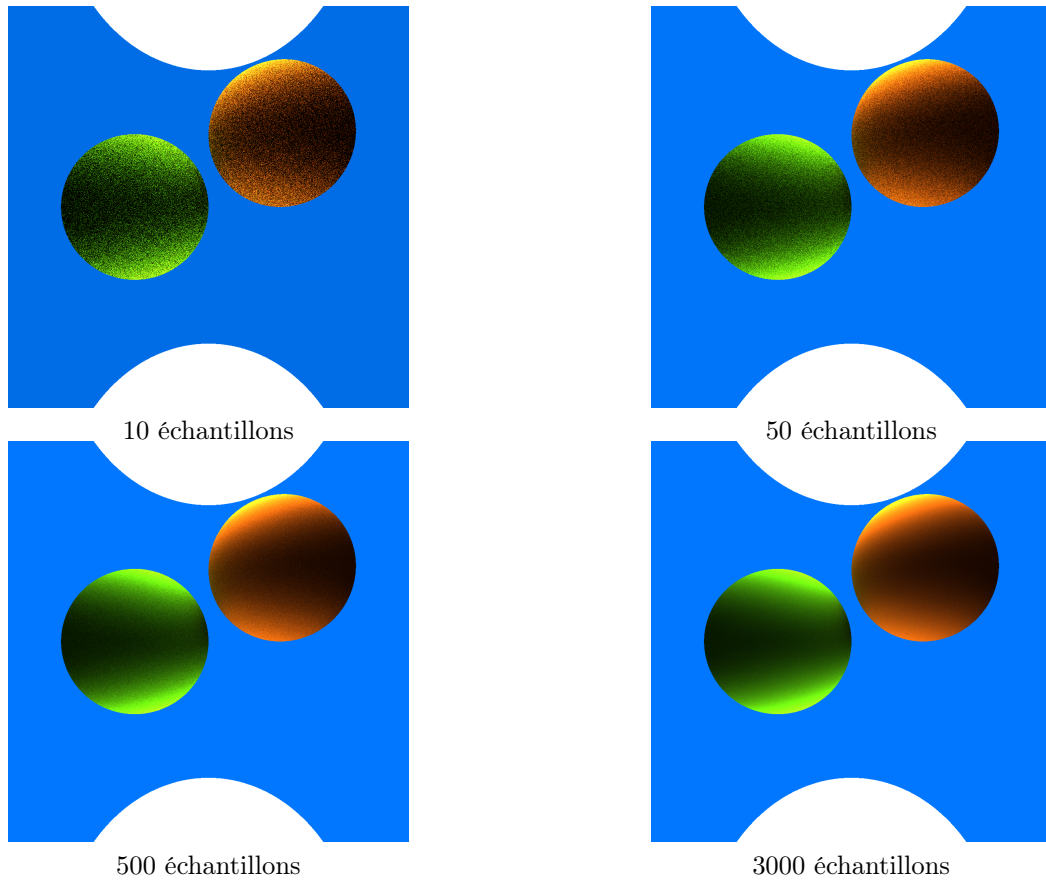


FIGURE 6.1 – Résultat d’une même scène pour différents nombres d’échantillons
 $(w, h) = (1000, 1000)$, 8 rebonds

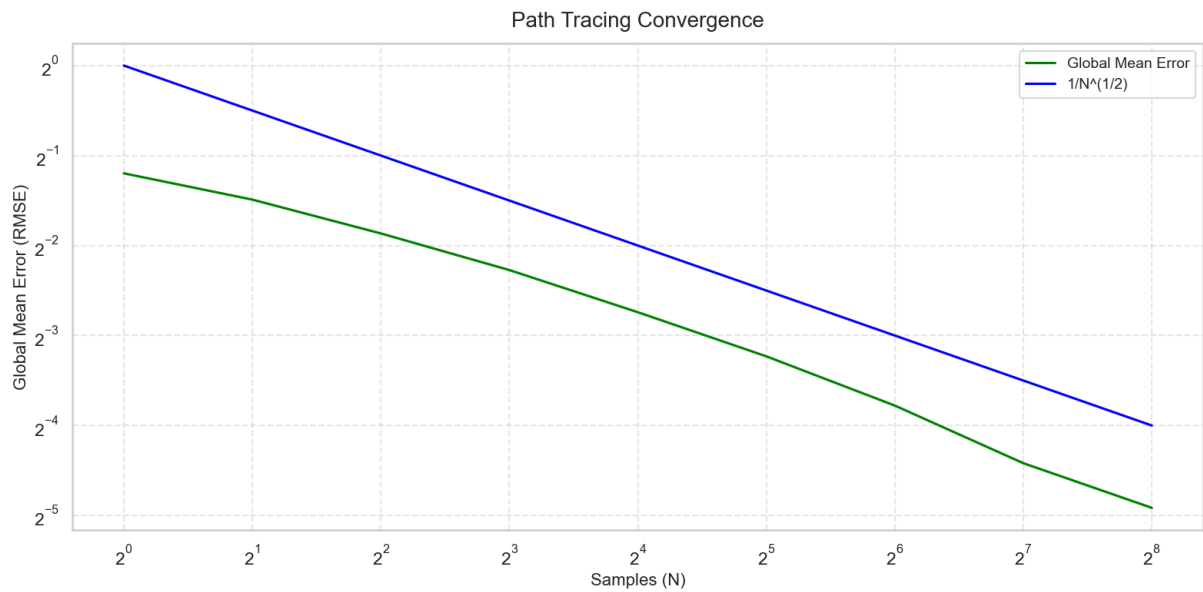


FIGURE 6.2 – Convergence de la Global Mean Error avec l’image de référence en fonction du nombre d’échantillons par rayon en échelle logarithmique. L’image de référence a comme nombre d’échantillons $N=16384$.
 $(w, h) = (1000, 1000)$, 8 rebonds

La méthode de Monte-Carlo est réputée comme ayant une convergence rapide dans un premier temps, puis une convergence lente pour un très grand nombre d'échantillons (cf. [Figure 6.1](#)).

Nous constatons cette convergence dans les résultats obtenus avec notre Path Tracer. En effet, la méthode de Monte-Carlo possède une convergence lente, en $O\left(\frac{1}{\sqrt{N}}\right)$, ce qui est visible avec notre version (cf [Figure 6.2](#)). Nous remarquons également une chute de l'erreur vers les dernières images puisque plus N augmente, plus l'image se rapproche de notre image de référence.

6.2 Analyse de performances

Dans cette dernière partie, nous allons discuter de l'implémentation actuelle et la manière dont nous allons analyser les performances.

6.2.1 Protocole de mesure

Pour analyser les performances, nous allons commencer par quantifier le temps d'exécution du programme en fonction du nombre d'échantillons par pixels. Ainsi, nous pourrions comparer le runtime en fonction de nos modifications.

Pour comparer les versions de notre Path Tracer, nous allons prendre un nombre d'échantillons fixé à 2000, puis nous comparerons les temps d'exécutions de chaque version.

6.2.2 Discussion de l'implémentation actuelle

La version naïve de notre Path Tracer converge vers la solution vraie. Cependant, elle possède plusieurs imprécisions. En effet, nous calculons la couleur d'un pixel pour N rayons, même si le rayon ne rencontre aucun obstacle sur son chemin, ce qui est une perte de temps, d'autant plus si la surface visible des objets par la caméra est inférieure à celle du ciel.

Une autre imprécision est due à l'appel récursif de la fonction RaySampling. Elle pourra facilement être remplacée par une boucle, permettant de mieux contrôler le comportement des rayons dans des cas spécifiques, comme celui énoncé plus tôt. De plus, contrairement à l'appel récursif, une boucle est facilement parallélisable, augmentant donc les performances de notre programme.

6.2.3 Possibles améliorations

A l'heure actuelle, toutes nos implémentations sont des versions naïves. On pourrait améliorer certaines zones critiques de notre code en évitant de faire beaucoup de division. Par exemple, si on a des divisions avec les mêmes valeurs, on pourrait les faire qu'une fois.

Il y a aussi la possibilité de limiter l'allocation de la mémoire dans la heap, et de maximiser l'utilisation de la mémoire statique (stack). Les pistes d'améliorations citées ci-dessus restent des améliorations mineures, mais on pourrait aller beaucoup plus loin. Par exemple, paralléliser la fonction ray sampling, en divisant le travail en plusieurs groupes (block).

On pourrait décider de la taille d'un groupe de travail de manière statique ou de manière dynamique. Pour cela, on pourrait utiliser openMP qui permet de gérer plusieurs types de parallélisations.

Il y a aussi la vectorisation, mais de ce qu'on sait, cela aura peu d'impacte sur les performances vu que le compilateur le fait de manière automatique.

7 Annexes

7.1 Démonstration de l'équation de rendu

7.1.1 Cas Général

Par définition, la radiance incidente se détermine comme suit :

$$L_o = L_e + L_r$$

L_r représente l'ensemble des rayons incidents sur la surface qui se réfléchit en un rayon de direction ω_i dont la formule est :

$$L_r = \int_{\Omega} f(x, \omega_o, \omega_i) L_i(x, \omega_i) \cos(\theta_i) d\omega_i$$

Ce qui nous donne l'équation de rendu ci-dessous :

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_{\Omega} f(x, \omega_o, \omega_i) L_i(x, \omega_i) \cos(\theta_i) d\omega_i$$

7.1.2 Matériaux Lambertiens

Les matériaux lambertiens ont les propriétés suivantes :

- Ils sont isotropes (leur structures sont identiques peu importe où l'on regarde)
- Ils sont diffus (les rayons réfléchis sont dans l'hémisphère Ω de manière équiprobable)
- Ils n'émettent pas de lumière dans le visible.

Les conséquences sont que pour un matériau lambertien, le BRDF est invariant de x et ω .

Les rayons de direction ω_i sont seulement dans l'hémisphère Ω .

De ces faits, nous pouvons réécrire l'équation de rendu :

$$L_o(x, \omega_o) = f_{const} \int_{\Omega} L_i(x, \omega_i) \cos(\theta_i) d\omega_i$$

On définit l'irradiance incidente $E_i = \int_{\Omega} L_i(x, \omega_i) \cos(\theta_i) d\omega_i$ et l'irradiance réfléchie $E_o = \int_{\Omega} L_o(x, \omega_o) \cos(\theta_i) d\omega_i$ la puissance d'un rayonnement électromagnétique frappant sur une unité de surface.

On définit également l'albédo $\rho(x, \omega_i, \omega_o)$ qui représente le rapport entre la radiance réfléchis et la radiance incidente : $\rho(x, \omega_i, \omega_o) = \frac{L_o(x, \omega_i, \omega_o)}{L_r(x, \omega_i, \omega_o)} = \frac{E_o(x, \omega_i, \omega_o)}{E_r(x, \omega_i, \omega_o)}$

Nous avons donc le système suivant :

$$\begin{cases} L_o = f_{const} \times E_i \\ E_o = L_o \int_{\Omega} \cos(\theta) d\omega \end{cases} \Leftrightarrow \begin{cases} L_o = f_{const} \times E_i \\ E_o = L_o \int_{\Omega} \cos(\theta) d\omega \end{cases}$$

En se plaçant en coordonnées sphériques pour un hémisphère de rayon 1 : $d\omega = \sin(\theta) d\theta d\varphi$ avec $\theta \in [0, \frac{\pi}{2}]$ et $\varphi \in [-\pi, \pi]$.

Donc $\int_{\Omega} \cos(\theta) d\omega$ devient :

$$\int_{\Omega} \cos(\theta) d\omega = \int_{-\pi}^{\pi} \int_0^{\frac{\pi}{2}} \cos(\theta) \sin(\theta) d\theta d\varphi = 2\pi \left[\frac{\sin^2(\theta)}{2} \right]_0^{\frac{\pi}{2}} = \boxed{\pi}$$

Donc le système devient :

$$\begin{cases} L_o = f_{const} \times E_i \\ L_o = \frac{E_o}{\pi} \end{cases}$$

Donc nous avons la formule du BRDF suivante :

$$\frac{E_o}{\pi} = f_{const} \times E_i \Leftrightarrow f_{const} = \frac{E_o}{E_i} \frac{1}{\pi}$$

$$\Leftrightarrow f_{const} = \boxed{\frac{\rho}{\pi}}$$

Ceci nous permet de conclure avec l'équation sur laquelle nous allons travailler dans cette partie du projet :

$$\boxed{L_o(x, \omega) = \frac{\rho}{\pi} \int_{\Omega} L_i(x, \omega_i) \cos(\theta_i) d\omega_i}$$

7.2 Démonstration sur les matrices de rotations

Dans le cadre de ce projet, nous fixons la caméra à une position $\mathbf{E} \in \mathbb{R}^3$ quelconque et nous posons être dans la base canonique de $\mathbb{R}^3$: $(\vec{r}_0, \vec{u}_0, \vec{d}_0) = \left(\begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \right)$ à ce point $\mathbf{E}$. Notons que jusqu'à maintenant la caméra pointe dans la direction de $\vec{d}_0$. En restant à cette position, nous allons implémenter la possibilité pour la caméra de pivoter dans n'importe quelle direction, nous cherchons donc ici à démontrer que l'application utilisée pour implémenter cela fonctionne pour n'importe quel angle choisi.

Pour réaliser cela, nous considérons un angle α présentant une rotation par rapport à l'axe formé par le vecteur $\vec{r}_0$ et un second angle β présentant une rotation par rapport à l'axe formé par le vecteur $\vec{u}_0$. L'application qui permet la rotation de la caméra est déterminée par la composition des deux applications suivantes $\forall \alpha, \beta \in \mathbb{R}$ (nous pouvons restreindre α et β à $[0, 2\pi]$ mais rien nous pose cette condition ici) :

$$\begin{aligned} \text{pitch}_{\alpha} : \mathbb{R}^3 &\rightarrow \mathbb{R}^3 & \text{et} & & \text{yaw}_{\beta} : \mathbb{R}^3 &\rightarrow \mathbb{R}^3 \\ (x, y, z) &\mapsto \text{pitch}_{\alpha}(x, y, z) & & & (x, y, z) &\mapsto \text{yaw}_{\beta}(x, y, z) \end{aligned}$$

Qui sont toutes deux définies grâce aux matrices de rotations sur leurs axes respectifs :

$$\begin{aligned} \forall \alpha \in \mathbb{R} : \text{pitch}_{\alpha}(x, y, z) &= \begin{pmatrix} 0 & 0 & 1 \\ \sin \alpha & \cos \alpha & 0 \\ \cos \alpha & -\sin \alpha & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} z \\ x \sin \alpha + y \cos \alpha \\ x \cos \alpha - y \sin \alpha \end{pmatrix} \\ \forall \beta \in \mathbb{R} : \text{yaw}_{\beta}(x, y, z) &= \begin{pmatrix} -\sin \beta & 0 & \cos \beta \\ 0 & 1 & 0 \\ \cos \beta & 0 & \sin \beta \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} -x \sin \beta + z \cos \beta \\ y \\ x \cos \beta + z \sin \beta \end{pmatrix} \end{aligned}$$

Nous nous intéressons donc à l'application : $\text{pitch}_{\alpha} \circ \text{yaw}_{\beta}$ appliquée à chacun des vecteurs formant la base canonique. Soient les vecteurs $(\vec{r}, \vec{u}, \vec{d}) \in \mathbb{R}^3$ définis tels que $\forall \alpha, \beta \in \mathbb{R}$:

$$\begin{cases} \text{pitch}_{\alpha} \circ \text{yaw}_{\beta}(\vec{r}_0) = \vec{r} \\ \text{pitch}_{\alpha} \circ \text{yaw}_{\beta}(\vec{u}_0) = \vec{u} \\ \text{pitch}_{\alpha} \circ \text{yaw}_{\beta}(\vec{d}_0) = \vec{d} \end{cases}$$

Nous voulons montrer que la famille de vecteurs $(\vec{r}, \vec{u}, \vec{d})$ forment une base orthonormée de $\mathbb{R}^3$. C'est-à-dire qu'après utiliser cette application sur la position de la caméra, celle-ci pointerait maintenant dans la direction de $\vec{d}$. Soient $\alpha, \beta \in \mathbb{R}$.

Tout d'abord écrivons l'expression de $\text{pitch}_{\alpha} \circ \text{yaw}_{\beta} \forall (x, y, z) \in \mathbb{R}^3$:

$$\text{pitch}_{\alpha} \circ \text{yaw}_{\beta}(x, y, z) = \begin{pmatrix} x \cos \beta + z \sin \beta \\ (-x \sin \beta + z \cos \beta) \sin \alpha + y \cos \alpha \\ (-x \sin \beta + z \cos \beta) \cos \alpha - y \sin \alpha \end{pmatrix}$$

Appliquons-la maintenant aux vecteurs de la base canonique :

$$\begin{cases} \vec{r} = \text{pitch}_\alpha \circ \text{yaw}_\beta (\vec{r}_0) = \begin{pmatrix} \cos \beta \\ -\sin \alpha \sin \beta \\ -\cos \alpha \sin \beta \end{pmatrix} \\ \vec{u} = \text{pitch}_\alpha \circ \text{yaw}_\beta (\vec{u}_0) = \begin{pmatrix} 0 \\ \cos \alpha \\ -\sin \alpha \end{pmatrix} \\ \vec{d} = \text{pitch}_\alpha \circ \text{yaw}_\beta (\vec{d}_0) = \begin{pmatrix} \sin \beta \\ \cos \beta \sin \alpha \\ \cos \alpha \cos \beta \end{pmatrix} \end{cases}$$

Montrons premièrement que chacun de ces vecteurs sont de norme 1 :

$$\begin{aligned} \|\vec{r}\|^2 &= \cos^2 \beta + \sin^2 \alpha \sin^2 \beta + \cos^2 \alpha \sin^2 \beta \\ &= \cos^2 \beta + \sin^2 \beta (\sin^2 \alpha + \cos^2 \alpha) \\ &= \cos^2 \beta + \sin^2 \beta \\ &= 1 \end{aligned}$$

$$\begin{aligned} \|\vec{u}\|^2 &= \cos^2 \alpha + \sin^2 \alpha \\ &= 1 \end{aligned}$$

$$\begin{aligned} \|\vec{d}\|^2 &= \sin^2 \beta + \cos^2 \beta \sin^2 \alpha + \cos^2 \alpha \cos^2 \beta \\ &= \sin^2 \beta + \cos^2 \beta (\sin^2 \alpha + \cos^2 \alpha) \\ &= \sin^2 \beta + \cos^2 \beta \\ &= 1 \end{aligned}$$

Calculons maintenant le produit vectoriel de $\vec{r}$ avec $\vec{u}$:

$$\begin{aligned} \vec{r} \wedge \vec{u} &= \begin{pmatrix} \sin^2 \alpha \sin \beta + \cos^2 \alpha \sin \beta \\ \cos \beta \sin \alpha \\ \cos \alpha \cos \beta \end{pmatrix} \\ &= \begin{pmatrix} \sin \beta (\sin^2 \alpha + \cos^2 \alpha) \\ \cos \beta \sin \alpha \\ \cos \alpha \cos \beta \end{pmatrix} \\ &= \begin{pmatrix} \sin \beta \\ \cos \beta \sin \alpha \\ \cos \alpha \cos \beta \end{pmatrix} \\ &= \vec{d} \end{aligned}$$

Nous remarquons que le vecteur $\vec{d}$ est normal au plan formé par $\vec{r}$ et $\vec{u}$. Il nous reste donc à montrer que $\vec{r}$ et $\vec{u}$ sont orthogonaux :

$$\begin{aligned} \vec{r} \cdot \vec{u} &= \cos \beta \times 0 - \sin \alpha \sin \beta \cos \alpha + \cos \alpha \sin \beta \sin \alpha \\ &= 0 \end{aligned}$$

Nous avons montré que $\vec{d}$ est normal à $\vec{r}$ et $\vec{u}$; que $\vec{r}$ et $\vec{u}$ sont orthogonaux entre eux et que les trois vecteurs ont pour norme 1. Cela montre donc que $(\vec{r}, \vec{u}, \vec{d})$ est une base orthonormée de $\mathbb{R}^3$, donc que l'application $\text{pitch}_\alpha \circ \text{yaw}_\beta$ établit une rotation de la base canonique $(\vec{r}_0, \vec{u}_0, \vec{d}_0)$ pour des $\alpha, \beta \in \mathbb{R}$ quelconques. Donc cette application nous permet bien de pivoter la caméra dans n'importe quelle direction, c'est-à-dire en considérant la sphère centrée en $\mathbf{E}$ de rayon $\|\vec{d}_0\| = 1$ incluse dans $\mathbb{R}^3$:

$\forall x \in (\mathbf{E}, \|\vec{d}_0\|), \exists \alpha, \beta \in \mathbb{R}$ tels que $\text{pitch}_\alpha \circ \text{yaw}_\beta (\vec{d}_0) = \vec{E}x$.

7.3 Parité des nombres de pixels dans la formation d'un écran

Afin d'obtenir des expressions de $\vec{d}_u$ et $\vec{d}_r$ correctes en général, nous devons nous assurer de la position de $\mathbf{C}$ par rapport au centre des pixels sur l'écran. Celle-ci dépend de la parité de (w, h) donnant quatre cas de figure :

Cas 1: w pair, h pair

$P_{0,0}$	.	.	.
.	.	$\mathbf{C}$	.

Cas 2: w pair, h impair

$P_{0,0}$	.	.	.
.	.	$\mathbf{C}$	.
.	.	.	.

Cas 3: w impair, h pair

$P_{0,0}$	.	.
.	.	.
.	$\mathbf{C}$	.
.	.	.

Cas 4: w impair, h impair

$P_{0,0}$	.	.	.
.	.	.	.
.	.	$\mathbf{C}$	.
.	.	.	.

FIGURE 7.1 – Position de $\mathbf{C}$ selon la parité de (w, h)
 $(w, h) = \{(4, 2); (4, 3); (3, 4); (4, 4)\}$

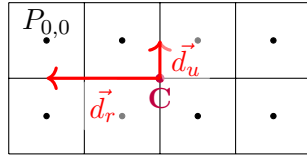
En choisissant :

$$\vec{d}_r = -2 \left(\frac{w-1}{h-1} \right) \tan\left(\frac{\theta}{2}\right) \frac{w-1}{2(w-1)} \vec{r} = -\left(\frac{w-1}{h-1} \right) \tan\left(\frac{\theta}{2}\right) \vec{r}$$

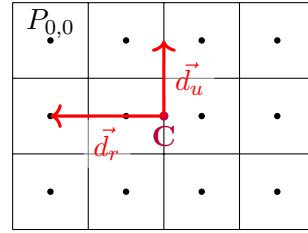
$$\vec{d}_u = 2 \tan\left(\frac{\theta}{2}\right) \frac{h-1}{2(h-1)} \vec{u} = \tan\left(\frac{\theta}{2}\right) \vec{u}$$

Nous pouvons les représenter pour chacun de ces cas en partant de $\mathbf{C}$, ils sont visuellement représentés tels que :

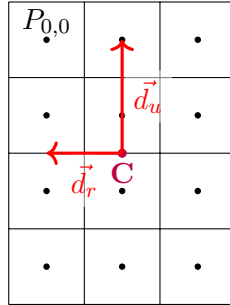
Cas 1: w pair, h pair



Cas 2: w pair, h impair



Cas 3: w impair, h pair



Cas 4: w impair, h impair

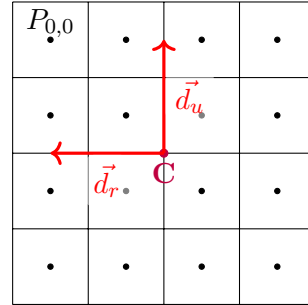


FIGURE 7.2 – Définition des vecteurs $\vec{d}_r$ et $\vec{d}_u$ indépendants de la parité de (w, h)
 $(w, h) = \{(4, 2); (4, 3); (3, 4); (4, 4)\}$

Cela généralise donc les distances nécessaires afin de déterminer par la suite les coordonnées de chaque centre des pixels de notre écran.

8 Bibliographie

Realistic Image Synthesis - Rendering Equation -, par Philipp Slusallek Karol Myszkowski Gurprit Singh : https://graphics.cg.uni-saarland.de/courses/ris-2019/slides/RIS01_RendEq.pdf

Importance Sampling of a Hemisphere, Universität Marburg : <https://www.mathematik.uni-marburg.de/~thormae/lectures/graphics1/code/ImportanceSampling/index.html>

Open-GL tutorial - Tutorial 13 - : Normal Mapping: <https://www.opengl-tutorial.org/fr/intermediate-tutorials/tutorial-13-normal-mapping/>

Physically Based Rendering <https://pbr-book.org/4ed/contents>

Generating Camera Rays with Ray-Tracing <https://www.scratchapixel.com/lessons/3d-basic-rendering/ray-tracing-generating-camera-rays/generating-camera-rays.html>

Ray Tracing in One Weekend <https://raytracing.github.io/books/RayTracingInOneWeekend.html>